

Binding, Directives & Pipes

everything that happens inside the template



The template is the HTML the user sees. Angular gives it superpowers: showing data, reacting to clicks, looping over lists, showing things conditionally, and formatting values. Today we cover binding (four kinds), the common directives (`*ngIf` , `*ngFor` , `ngSwitch`), and pipes.

1 Data binding: the four kinds

Binding connects your class (the data) and your template (the view). There are four kinds, and the brackets tell you the direction.

1. Interpolation: `{{ }}`

Shows a value from the class in the page. Class to view, read only.

```
<h1>Hello {{ name }}</h1>

// class: name = "Angular"; -> shows "Hello Angular"
```

2. Property binding: `[ ]`

Sets an element's property from a class value. Also class to view.

```
<img [src]="imageUrl" />
<button [disabled]="isBusy">Save</button>
```

Here `[src]` takes its value from the `imageUrl` property in the class. Note the difference: `{{ }}` puts text between tags, `[ ]` sets a property on the tag.

3. Event binding: `( )`

Runs a class method when something happens, like a click. View to class.

```
<button (click)="save()">Save</button>

// class:
save() { console.log("saved!"); }
```

4. Two-way binding: `[( )]`

Connects an input box to a class property **both ways** at once. Type in the box, the property updates. Change the property, the box updates. The `[( )]` shape is nicknamed "banana in a box".

```
<input [(ngModel)]="username" />
<p>You typed: {{ username }}</p>
```

Note: two-way binding with `ngModel` needs `FormsModule`. In an older module project, add it to the module's imports. In a standalone component, add `FormsModule` to the component's imports.

The four, in one look: `{{ }}` show a value · `[ ]` set a property · `( )` handle an event · `[[ ]]` both ways. Square = data in, round = event out, both together = two-way.

2 Directives: `*ngIf`, `*ngFor`, `ngSwitch`

Directives are special instructions you put on an element to change how it behaves. The three you will use constantly:

`*ngIf`: show or hide

Shows an element only if a condition is true. If false, the element is not on the page at all.

```
<p *ngIf="isLoggedIn">Welcome back!</p>
<p *ngIf="!isLoggedIn">Please log in.</p>
```

`*ngFor`: loop over a list

Repeats an element once for each item in an array. This is how you show lists.

```
<ul>
  <li *ngFor="let fruit of fruits">{{ fruit }}</li>
</ul>

// class: fruits = ["Apple", "Mango", "Banana"];
```

Angular creates one `<li>` per fruit. Three fruits, three list items. Change the array, the list changes with it.

`ngSwitch`: pick one of many

Shows one block out of several, based on a value. Like a switch statement in the template.

```
<div [ngSwitch]="role">
  <p *ngSwitchCase="'admin'">You are an admin</p>
  <p *ngSwitchCase="'user'">You are a user</p>
  <p *ngSwitchDefault>Unknown role</p>
</div>
```

If `role` is `"admin"`, only the first line shows. If it matches nothing, the default shows.

3 Pipes: format a value for display

A **pipe** transforms a value in the template, without changing the original data. You use it with the `|` symbol. Great for formatting dates, text, numbers, and currency.

```
<p>{{ name | uppercase }}</p>      <!-- ANGULAR -->
<p>{{ price | currency:'INR' }}</p> <!-- Rs 500.00 -->
<p>{{ today | date:'longDate' }}</p> <!-- August 5, 2026 -->
```

Built-in pipes you will use: `uppercase`, `lowercase`, `titlecase`, `date`, `currency`, `number`, `percent`, `json`, `slice`. You can also chain them: `{{ name | slice:0:5 | uppercase }}`.

4 Custom pipes: make your own

When no built-in pipe does what you want, write your own. A pipe is a class with the `@Pipe` decorator and a `transform` method that takes a value and returns the changed value.

DOUBLE.PIPE.TS

```
import { Pipe, PipeTransform } from "@angular/core";

@Pipe({ name: "double" })
export class DoublePipe implements PipeTransform {
  transform(value: number): number {
    return value * 2;
  }
}
```

USING IT IN A TEMPLATE

```
<p>{{ 10 | double }}</p>    <!-- shows 20 -->
```

The `transform` method receives the value on the left of the `|`, does its work, and returns the result. Here `10` comes in, `20` goes out. You can also make a pipe with the CLI: `ng generate pipe double`.

A pipe can take arguments too, written after a colon:

```
@Pipe({ name: "multiply" })
export class MultiplyPipe implements PipeTransform {
  transform(value: number, times: number): number {
    return value * times;
  }
}

<!-- template -->
<p>{{ 10 | multiply:3 }}</p>    <!-- shows 30 -->
```

5 Remember this

Binding, four kinds: `{{ }}` show a value, `[ ]` set a property, `( )` handle an event, `[( )]` two-way with `ngModel`.

***ngIf** shows or hides. ***ngFor** loops over a list. **ngSwitch** picks one block of many.

Pipes format a value for display with `|`, without changing the data.

Custom pipe: a class with `@Pipe` and a `transform` method that returns the changed value.

6 Homework

Try all four bindings: show a name with `{{ }}`, set an image with `[src]`, handle a button `(click)`, and bind an input with `[(ngModel)]`.

Loop a list with `*ngFor`, and show or hide a message with `*ngIf`.

Use two pipes on some text, like `uppercase` and `date`.

Write a custom pipe that doubles a number, and use it in a template.